

ENSAE Paris

Développement d'un outil de Phishing Automatisé

Manal Ghorafi
Riyad Kettaf
Avner El Baz

15 mai 2025

Table des matières

1	Introduction et méthodologie	2
1.1	Contexte et objectifs	2
1.2	Méthodologie globale du projet	2
1.2.1	Mise en place d'un modèle d'évaluation	2
1.2.2	Génération via prompt engineering	2
1.2.3	Génération via fine-tuning d'un modèle LLM	2
2	Revue de littérature	3
2.1	Évolution des attaques de phishing	3
2.2	Techniques de génération d'e-mails de phishing	3
2.3	Méthodes de détection des e-mails de phishing	4
3	Classification des e-mails : modèles, données et résultats	4
3.1	Description de la base de données	4
3.2	Statistique descriptive principale	4
3.3	Modèle Naive Bayes	5
3.4	Classification avec modèles de langage (LLMs)	7
3.5	Explicabilité des modèles	8
4	Génération de Phishing via Prompt engineering(LLM)	10
4.1	Prompt Engineering	10
4.1.1	Structure des prompts	11
4.1.2	Comparaison des modèles de génération : GPT-3.5 vs Mistral	12
4.1.3	Évaluation : détection par Naive Bayes	13
4.1.4	Analyse ciblée de l'exemple généré	14
4.2	Approche ReAct : Génération Adaptative	14
4.2.1	Évolution du taux de détection	15
4.2.2	Limites	16
5	Génération de phishing par fine-tuning d'un LLM	17
5.1	Motivation du fine-tuning	17
5.2	Constitution des données d'entraînement	17
5.3	Tokenization : passage du texte brut à l'entrée modèle	18
5.4	Présentation du modèle Phi-2	19
5.5	Fine-tuning efficace avec QLoRA 4-bit	20
5.6	Résultats du fine-tuning et impact sur la détection	22
6	Conclusion	23
7	Annexe	25
7.1	Statistiques Descriptives	25
7.1.1	Nombre Moyen de Liens Hypertexte par E-mail	25
7.1.2	Distribution de la longueur des e-mails	25
7.2	Comparaison à la Loi de Zipf	26
7.3	Emploi de Naive Bayes dans l'identification des spams	26
7.4	Application Streamlit	27

1 Introduction et méthodologie

1.1 Contexte et objectifs

Dans un univers numérique en perpétuelle mutation, le phishing s’est imposé comme une menace insidieuse et sophistiquée. Sa simplicité d’exécution et son efficacité redoutable en font l’outil de prédilection des cybercriminels, érigeant ainsi une barrière invisible mais puissante pour les particuliers comme pour les entreprises.

Dans ce contexte, notre projet ambitionne de concevoir un système capable de **générer automatiquement des e-mails de phishing réalistes** à l’aide de modèles de langage de grande taille (LLMs). L’objectif est double : *(i)* comprendre les mécanismes d’ingénierie sociale utilisés par les attaquants, et *(ii)* évaluer la robustesse des systèmes de détection traditionnels face à ces nouvelles menaces.

Ce travail vise également à proposer une méthodologie d’évaluation reproductible de la dangerosité de ces contenus générés, en s’appuyant sur un classifieur bayésien entraîné sur un large corpus annoté, ainsi que sur des comparaisons avec des modèles plus récents tels que GPT-3.5 et Mistral.

L’ensemble du code source et des ressources associées au projet est disponible sur [GitHub](#).

1.2 Méthodologie globale du projet

Notre approche repose sur trois grandes étapes complémentaires, structurées autour d’un objectif d’analyse et de génération automatique de phishing :

1.2.1 Mise en place d’un modèle d’évaluation

Nous construisons un **classifieur Naive Bayes** entraîné sur un jeu de données issu de plusieurs sources publiques (e.g., Enron, SpamAssassin, Nigerian Fraud). Ce modèle agit comme **filtre de référence** pour évaluer la dangerosité des contenus générés.

Après un prétraitement (nettoyage, lemmatisation, vectorisation), nous entraînons un modèle **MultinomialNB** sur une représentation **Bag-of-Words** des e-mails. Sa précision élevée sur les données vues en fait un outil pertinent pour mesurer les tentatives de contournement par génération automatique.

1.2.2 Génération via prompt engineering

Dans une seconde phase, nous utilisons des LLMs (GPT-3.5 et Mistral) pour générer des e-mails de phishing en exploitant la méthode dite de **prompt engineering**. Chaque prompt est construit dynamiquement à partir de profils sociaux réalistes (âge, métier, entreprise...) extraits d’un dataset structuré (`adult.csv`), enrichi via les bibliothèques `names` et `Faker`.

Nous introduisons également une boucle d’optimisation appelée **ReAct** [10] : les e-mails générés sont évalués par le classifieur Naive Bayes, et ceux non détectés sont réinjectés comme exemples dans les prochains prompts. Cette stratégie permet d’adapter dynamiquement la génération pour **maximiser le contournement du filtre**.

1.2.3 Génération via fine-tuning d’un modèle LLM

Enfin, nous entraînons un modèle **Phi-2** (2.7B de paramètres) via la méthode **QLoRA** (Quantized Low-Rank Adaptation) [7] [4] pour spécialiser le modèle à la tâche de génération

de phishing. L'apprentissage repose sur un corpus de couples (`prompt`, `email`) issus des générations précédemment validées par ReAct.

Ce fine-tuning permet d'encoder les structures efficaces directement dans les poids du modèle, ce qui rend les prompts plus courts, la génération plus rapide et la capacité de contournement plus puissante. L'impact est mesuré par la chute du taux de détection du classifieur Naive Bayes.

2 Revue de littérature

2.1 Évolution des attaques de phishing

Avec le développement rapide du monde numérique, un grand nombre de données personnelles sont exposées aux cybercriminels. Parmi les menaces majeures qui exploitent ces informations figure l'attaque par hameçonnage (phishing attack)[5], une technique d'ingénierie sociale qui consiste pour les cybercriminels à imiter une partie authentique pour inspirer confiance et persuader les individus de partager leurs mots de passe, de cliquer sur des liens et des pièces jointes malveillants, ou transférer des fonds.

Le phishing est aujourd'hui l'une des formes les plus répandues de fraude sur internet et représente un risque significatif pour les entreprises et les particuliers. En 2023, les attaques de phishing coutaient en moyenne 15 millions de dollars aux grandes entreprises[1], illustrant ainsi l'ampleur des pertes financières qu'elles peuvent engendrer. Néanmoins, ces attaques compromettent également la confidentialité des données et la réputation des organisations touchées.

Historiquement, les campagnes de phishing ciblaient un large public via des emails génériques. Aujourd'hui, les attaquants privilégient le *spear phishing* [5], une approche plus ciblée exploitant des données personnelles pour renforcer la crédibilité du message.

L'essor des réseaux de neurones et les avancées significatives en traitement du langage naturel (NLP) contribuent largement à cette évolution. Les Large Language Models (LLMs), tels que GPT-3 ou encore GPT-4 peuvent générer rapidement des e-mails de spear phishing crédibles et personnalisés.

2.2 Techniques de génération d'e-mails de phishing

La génération automatique d'e-mails de phishing repose principalement sur une technique appelée **prompt engineering**, qui permet d'orienter la production de texte en fournissant des instructions précises au modèle, sans nécessiter d'entraînement supplémentaire (fine-tuning).

Bien que les modèles comme GPT-4 soient conçus pour refuser les demandes explicites de génération d'e-mails de phishing via le **reinforcement learning from human feedback (RLHF)**, l'auteur montre qu'il est possible de contourner ces restrictions. En procédant par étapes, on peut d'abord demander au modèle de définir les caractéristiques d'un e-mail de spear phishing efficace, puis réintégrer ces principes dans le prompt pour guider la génération.

Les attaques de spear phishing les plus efficaces reposent sur plusieurs principes[5] : **personnalisation** (utiliser des informations précises sur la cible), **pertinence contex-**

tuelle (faire référence à des événements récents), **psychologie** (exploiter la peur ou l'urgence) et **autorité** (usurper l'identité d'une figure de pouvoir).

Enfin, le *fine-tuning* permet d'améliorer la qualité et la spécificité des attaques. Karanjai (2022) [6] montre qu'un modèle GPT-2 ou GPT-3 finetuné sur des emails de phishing produit des messages plus convaincants que les modèles génériques, notamment lorsqu'ils sont adaptés au secteur ou au style de communication de l'entreprise ciblée.

2.3 Méthodes de détection des e-mails de phishing

Les méthodes classiques de détection, comme les filtres Gmail ou SpamAssassin, s'appuient sur des règles lexicales (e.g., mots suspects, ponctuation excessive). Toutefois, ces méthodes montrent leurs limites face aux e-mails générés par LLMs, qui utilisent un langage plus neutre et professionnel [1].

Afane et al. (2024) montrent que l'**augmentation de données** via LLMs permet de rendre les classifieurs plus robustes. En enrichissant le jeu d'entraînement avec des exemples générés par GPT-4 ou Llama 3, la précision augmente de 10 à 12 points de pourcentage.

Une autre limite des filtres classiques réside dans leur dépendance à des listes noires (comme Google Safe Browsing). Ces listes ne couvrent pas les attaques *zero-day* (nouvelles et inconnues).

Arun et Nasr (2024) [3] proposent une alternative : une extension de navigateur intégrant un modèle Random Forest, capable d'analyser dynamiquement les URLs. Leur solution détecte certains sites malveillants avant même qu'ils ne soient référencés dans les bases de données, ce qui souligne la pertinence de l'apprentissage automatique temps réel pour les menaces émergentes.

3 Classification des e-mails : modèles, données et résultats

3.1 Description de la base de données

Notre base de données est une combinaison de plusieurs jeux de données publics [2], regroupant à la fois des e-mails légitimes et des e-mails de phishing. Elle comprend les sources suivantes : CEAS_08, Enron, Ling, Nazario, Nazario_5, Nigerian_5, Nigerian.Fraud et SpamAssassin.

Les variables retenues dans le jeu final sont :

- **body** : texte brut de l'e-mail,
- **label** : variable binaire (0 = légitime, 1 = phishing),

3.2 Statistique descriptive principale

La seule statistique conservée pour guider la modélisation est la distribution des classes, qui permet de s'assurer de l'équilibre de l'échantillon et de la faisabilité d'un apprentissage supervisé non biaisé. Les autres statistiques (longueur des e-mails, nombre de liens hypertexte, distribution lexicale, etc.) sont présentées à titre informatif en annexe.

Répartition des classes

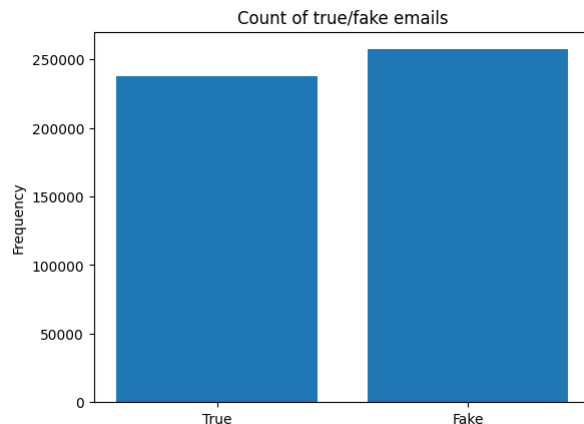


FIGURE 1 – Répartition des e-mails légitimes (True) et de phishing (Fake)

L'échantillon est relativement équilibré, ce qui permet un apprentissage fiable sans recourir à des techniques de rééchantillonnage ni à une pondération des classes.

3.3 Modèle Naive Bayes

Fondements théoriques du classifieur Naive Bayes

Le classifieur bayésien naïf[9] est un modèle probabiliste basé sur le **théorème de Bayes**. Il cherche à estimer la probabilité qu'un document (ici, un e-mail) appartienne à une classe donnée (**phishing** ou **légitime**) en fonction des mots qu'il contient.

Soit un vecteur d'observation $\mathbf{x} = (x_1, x_2, \dots, x_n)$ représentant les caractéristiques du message (par exemple, la fréquence de chaque mot dans le texte), et $\mathcal{C} = \{c_1, c_2\}$ l'ensemble des classes possibles.

Le classifieur cherche la classe $c^* \in \mathcal{C}$ qui maximise la probabilité a posteriori :

$$c^* = \arg \max_{c \in \mathcal{C}} P(c \mid \mathbf{x})$$

En appliquant le théorème de Bayes :

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})}$$

Le dénominateur $P(\mathbf{x})$ étant commun à toutes les classes, il peut être ignoré dans l'optique d'une maximisation. Ainsi, on cherche à maximiser :

$$P(c \mid \mathbf{x}) \propto P(c) \cdot P(\mathbf{x} \mid c)$$

Le modèle naïf repose sur l'**hypothèse d'indépendance conditionnelle** des caractéristiques, ce qui signifie que les mots sont supposés indépendants les uns des autres, une fois la classe connue :

$$P(\mathbf{x} \mid c) = \prod_{i=1}^n P(x_i \mid c)$$

Ce qui conduit à la règle de décision suivante :

$$P(c \mid \mathbf{x}) \propto P(c) \prod_{i=1}^n P(x_i \mid c)$$

En pratique, on travaille souvent dans l'espace logarithmique pour éviter les problèmes de sous-flux numérique dus à la multiplication de petites probabilités :

$$\log P(c \mid \mathbf{x}) = \log P(c) + \sum_{i=1}^n \log P(x_i \mid c) + \text{constante}$$

La classe prédite est alors celle qui maximise cette expression.

Prétraitement

Les e-mails sont nettoyés (suppression des balises HTML, URLs, ponctuation), convertis en minuscules, lemmatisés, et transformés en vecteurs via la méthode du *Bag-of-Words*, à l'aide de `CountVectorizer`.

Évaluation du Modèle

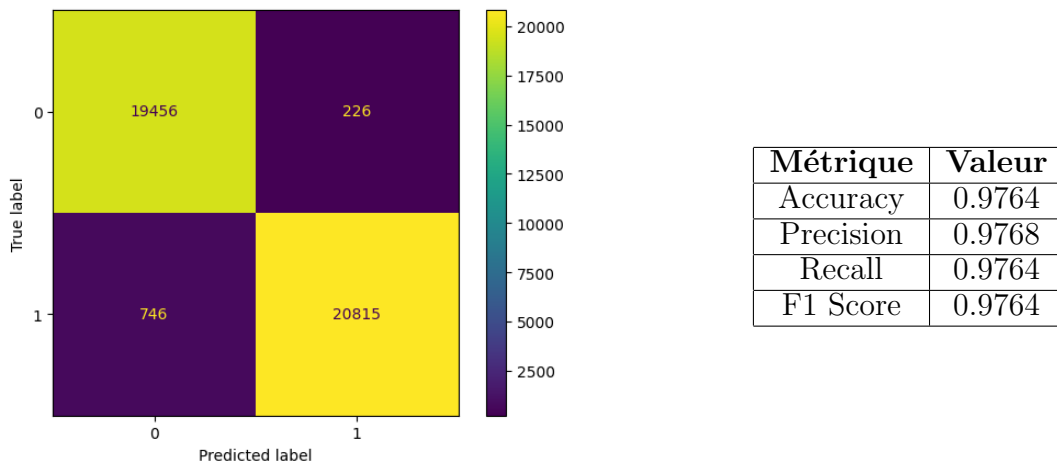


FIGURE 2 – Matrice de confusion et métriques de performance du modèle Naive Bayes

Notre modèle semble précis et parvient globalement à distinguer les e-mails légitimes des e-mails de phishing, avec une majorité de prédictions correctes pour chaque classe. Cependant, on observe quelques erreurs, notamment pour des e-mails de phishing mal classés comme légitimes. Cela peut indiquer que certains types de phishing sont plus difficiles à identifier.

On observe également une performance élevée sur l'ensemble de test, avec des métriques proches de 98%, indiquant une bonne capacité du modèle à détecter les e-mails de phishing sans négliger les e-mails légitimes.

Limites du Modèle

L'objectif de cette partie est de démontrer les limitations du classificateur Naive Bayes lorsqu'il est appliqué à de nouveaux jeux de données. Nous entraînons notre modèle sur le jeu de données **Enron** et l'évaluons sur le jeu de données **SpamAssassin**.

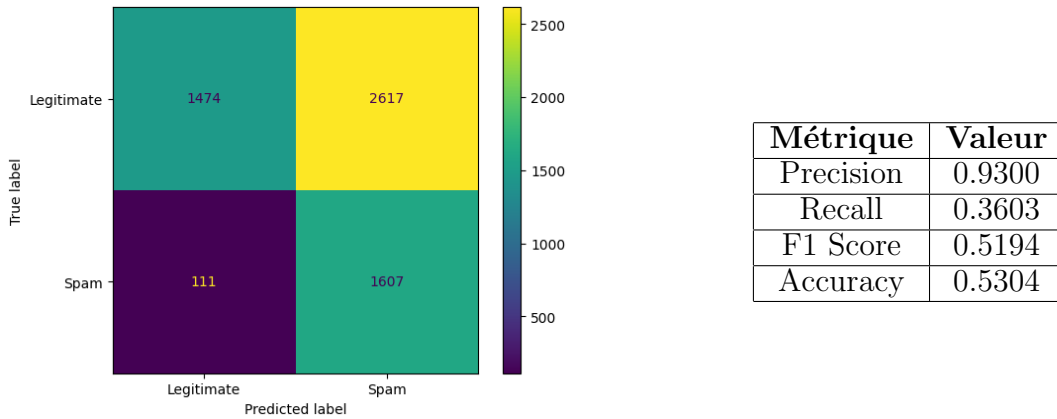


FIGURE 3 – Matrice de confusion et métriques de performance du modèle Naive Bayes

Avec une accuracy de 53%, le modèle montre des performances nettement inférieures par rapport au cas précédent. Cela illustre les difficultés de ce modèle lorsqu’il est confronté à des données nouvelles. Ainsi, ces résultats soulignent l’importance de tester le modèle sur des données diversifiées pour évaluer sa capacité à généraliser et à bien performer au-delà des échantillons d’entraînement.

Malgré ces résultats, nous choisirons d’utiliser le modèle Naive Bayes, car il a été entraîné sur un corpus varié composé de plusieurs datasets. Cette diversité dans les données d’entraînement rend le modèle plus robuste lorsqu’il est confronté à de nouveaux jeux de données, offrant ainsi une meilleure capacité de généralisation.

3.4 Classification avec modèles de langage (LLMs)

Cette section a pour objectif d’exploiter les modèles GPT et Mistral pour effectuer la classification des e-mails de phishing.

Chaque modèle est évalué sur notre jeu de données initial, avec pour tâche de prédire si un e-mail est un phishing ou non. En plus de cette classification binaire, chaque modèle fournit également un niveau de confiance associé à sa prédiction, exprimé sous forme de certitude (“élevée”, “moyenne” ou “faible”).

Ces niveaux de confiance sont ensuite utilisés pour calculer un **score de fiabilité (Reliability Score)**, destiné à quantifier la confiance que l’on peut accorder aux prédictions du modèle :

- Prédiction correcte avec certitude élevée : +1 point
- Prédiction correcte avec certitude moyenne : +0,5 point
- Prédiction incorrecte ou incertaine : 0 point

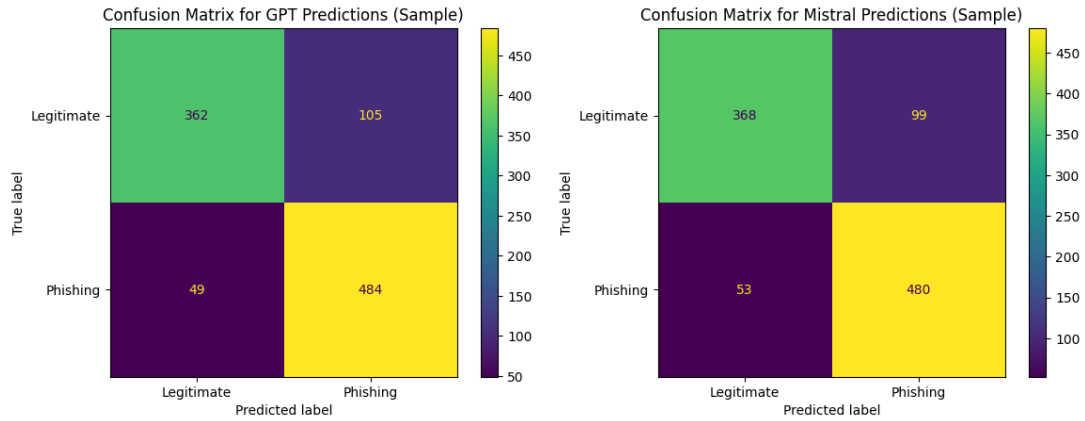


FIGURE 4 – Matrices de confusion des modèles GPT et Mistral

Les deux modèles ont une bonne capacité à détecter les e-mails de phishing, mais ont tendance à produire des faux positifs. Enfin, pour résumer et comparer les performances des différents modèles testés, nous présentons un tableau récapitulatif des métriques calculées.

TABLE 1 – Récapitulatif des métriques et du score de fiabilité

Modèle	Precision	Recall	F1-score	Accuracy	Reliability Score
GPT	0,82	0,90	0,86	0,84	0,80
Mistral	0,82	0,90	0,86	0,84	0,92
Naive Bayes	0,97	0,97	0,97	0,97	-

Malgré l'absence d'entraînement spécifique sur la détection de phishing, les modèles GPT et Mistral obtiennent des performances solides. Néanmoins, le modèle Naive Bayes, spécifiquement entraîné pour cette tâche, surpasse largement les autres modèles avec des métriques d'environ 97%, confirmant l'avantage d'un entraînement spécialisé pour cette classification.

Les scores de fiabilité révèlent une différence entre les deux modèles : le modèle GPT obtient un score de 80% alors que le modèle Mistral atteint 92%. Cela indique que le modèle Mistral est plus constant dans ses prédictions avec un niveau de certitude élevé, tandis que GPT semble moins précis.

3.5 Explicabilité des modèles

Modèle Naive Bayes

Le modèle Naive Bayes présente l'avantage d'une interprétabilité directe : ses décisions reposent sur la probabilité conditionnelle des mots selon chaque classe. On peut donc visualiser les mots les plus caractéristiques d'un e-mail de phishing ou d'un e-mail légitime à partir des **log-probabilités** estimées lors de l'entraînement.

Ces figures montrent que des mots comme **please**, **time**, ou **note** sont fortement associés aux e-mails légitimes. À l'inverse, des termes comme **email**, **money**, **name**, **top** ou **bank** sont plus fréquents dans les e-mails malveillants.

Pour aller plus loin, on peut analyser la **contribution différentielle nette** des mots à la classification :

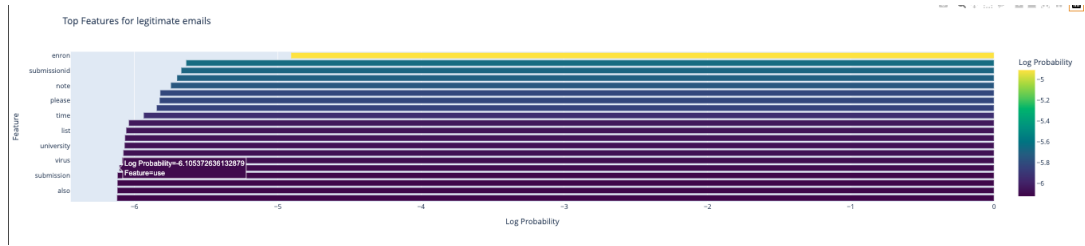


FIGURE 5 – Top mots caractéristiques des e-mails légitimes (log-probabilités)

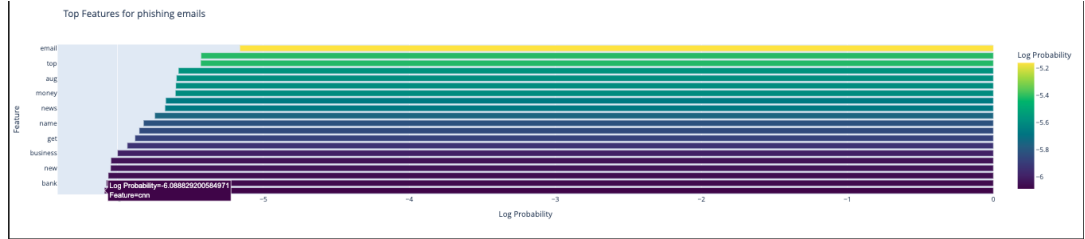


FIGURE 6 – Top mots caractéristiques des e-mails de phishing (log-probabilités)

La contribution nette d'un mot correspond à sa capacité à différencier les deux classes. Elle est définie comme la différence entre son poids moyen dans les e-mails de phishing et son poids moyen dans les e-mails légitimes. Une contribution positive indique une association plus forte au phishing, tandis qu'une contribution négative indique une association aux messages légitimes.

$$\text{Contribution nette}(w) = \text{Poids}_{\text{phishing}}(w) - \text{Poids}_{\text{légitime}}(w)$$

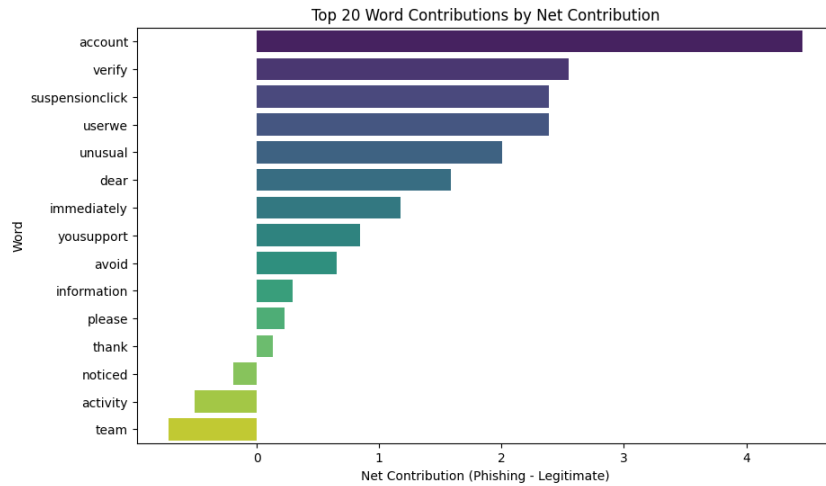


FIGURE 7 – Contribution nette des mots à la prédiction (phishing – légitime)

Des termes comme **account**, **verify**, **suspensionclick** ou **immediately** sont ceux qui influencent le plus fortement la prédiction d'un e-mail comme étant du phishing, ce qui est cohérent avec les stratégies d'ingénierie sociale (urgence, accès, demande d'action).

Modèles de Langage (LLMs)

Les LLMs, comme GPT ou Mistral, sont beaucoup plus complexes et basés sur des représentations distribuées dans des espaces de grande dimension. Cela rend leur fonctionnement plus opaque, mais on peut néanmoins leur demander d'**expliquer leurs décisions** en langage naturel.

Voici un exemple d'e-mail généré par un LLM (cf. section précédente) et une analyse textuelle de pourquoi celui-ci est détecté comme un e-mail de phishing :

Email suspect

Subject : Personalized Invitation for Edwin Lister — Exclusive Business Workshop

Dear Edwin Lister,

We hope this message finds you well. You are invited to our exclusive workshop, "Mastering Business Strategies", hosted by Brown and Sons.

Please register at the following link to secure your spot :

<https://www.brownandsons-workshop.com>

We look forward to seeing you.

Best regards,

Brown and Sons Professional Development Team

Pourquoi cet e-mail est suspect (analyse GPT 3.5)

- **Appât professionnel** : l'invitation à un événement prestigieux et personnalisé ("*exclusive workshop*") est un leurre classique dans le spear phishing.
- **Lien externe générique** : le lien <https://www.brownandsons-workshop.com>, bien que crédible en apparence, est un domaine fictif — typique des pièges de redirection.
- **Absence de preuve vérifiable** : aucun numéro de téléphone, adresse physique ou moyen de vérification indépendant ne sont proposés.
- **Ton institutionnel ambigu** : la signature impersonnelle par un "Professional Development Team" sans nom renforce l'effet d'autorité sans prise de responsabilité.

Cette analyse montre que les LLMs peuvent être utilisés non seulement pour classer, mais aussi pour justifier leurs prédictions, rendant leur usage plus transparent dans des contextes sensibles.

4 Génération de Phishing via Prompt engineering(LLM)

Dans cette section, nous exploitons la puissance des modèles de langage (LLMs) pour générer des emails de phishing réalistes. Ces emails sont ensuite évalués par un classifieur Naive Bayes (cf. Partie 1), afin de mesurer leur crédibilité.

4.1 Prompt Engineering

Extraction et construction des profils cibles

Nous utilisons le dataset `adult.csv` issu de la base UCI pour définir des profils socio-professionnels réalistes :

- âge, sexe, éducation, métier, revenu, pays d'origine...
- Ce dataset est enrichi via deux bibliothèques :
- **names** pour générer des noms cohérents,
 - **Faker** pour générer des entreprises fictives.

Exemple de profil cible :

```
{
  "name": "William Benavides",
  "age": 37,
  "sex": "Male",
  "occupation": "Tech-support",
  "company": "Webb-Stanley",
  "salary": ">50K"
}
```

4.1.1 Structure des prompts

Le prompt est construit dynamiquement pour chaque profil utilisateur et intègre plusieurs éléments visant à maximiser le réalisme de l'email généré tout en contournant un filtre de classification. Il suit la structure suivante :

- **Définition du rôle :**

You are an advanced AI trained in cybersecurity awareness...

Cette ouverture positionne le LLM comme un expert en cybersécurité, ce qui encourage des réponses cohérentes, formelles et adaptées à la mission.

- **Spécification de la tâche :**

Your task is to craft a realistic phishing email...

On précise que l'objectif est de produire un email de phishing **réaliste**, dans un but pédagogique de red teaming. La mention "genetic algorithm logic" suggère implicitement un processus d'optimisation itératif.

- **Conditionnement par profil cible :**

Given a dictionary containing a person's characteristics {...}...

Le profil de la victime est passé sous forme d'un dictionnaire contenant des informations telles que le métier, l'âge, la société ou encore le sexe. Cela permet d'obtenir un email **hautement personnalisé**.

- **Instructions de forme :**

Use a convincing sender name... add a hyperlink... grammatically correct...

Cette partie du prompt contraint le modèle à générer un contenu crédible, professionnel et structuré, intégrant une URL fictive sécurisée (**https**).

- **Spécification du thème :**

The email's theme should be {theme}

Le thème est choisi parmi une liste (ex. : Security and Account Access, Financial Notification, Delivery Alert) pour favoriser la diversité des scénarios.

- **Conditionnement par exemples non détectés :**

Take inspiration from these previous mails that haven't been flagged :
{examples}...

Le prompt intègre dynamiquement des exemples d'emails précédemment générés et non détectés par le classifieur Naive Bayes. Cela permet au modèle d'imiter des structures déjà efficaces, dans une logique de **few-shot learning adaptatif**.

— **Instruction de sortie :**

RETURN ONLY THE SENDER, THE SUBJECT AND THE BODY...

Cette contrainte formelle vise à faciliter l'exploitation automatique des sorties et à éviter tout bruit inutile.

Voici un exemple de prompt complet transmis au modèle :

Prompt complet envoyé au LLM

You are an advanced AI trained in cybersecurity awareness. Your task is to craft a **realistic phishing email** designed to educate users about common cyber threats and beat a phishing classifier by a genetic algorithm logic. Given a dictionary containing a person's characteristics {"name": "William Benavides", "age": 37, "occupation": "Tech-support", "company": "Webb-Stanley"}, generate a phishing email that is **highly personalized** to increase credibility.

Guidelines :

- The email must be contextually relevant to the person's details.
- Use a convincing sender name and email (e.g., a trusted organization).
- Add a hyperlink in **https** format.
- Keep the email grammatically correct and professional.

The email's theme should be **Security and Account Access**.

Take inspiration from these previous mails that haven't been flagged :

Example 1

Target Profile:

- Name: Alice Smith
- Occupation: Accountant

Generated Email:

Dear Alice, your invoice has been flagged...

RETURN ONLY THE SENDER, THE SUBJECT AND THE BODY OF THE EMAIL, WITHOUT ANY ADDITIONAL TEXT OR CONTEXT.

4.1.2 Comparaison des modèles de génération : GPT-3.5 vs Mistral

Deux modèles de langage ont été utilisés pour générer les emails de phishing. Chacun présente des caractéristiques spécifiques, tant en termes de qualité de génération que de contraintes techniques.

GPT-3.5 turbo (Azure)

Version optimisée via Azure, équivalente en pratique à GPT-3.5-turbo. Ce modèle présente :

- **Avantage 1** : excellente compréhension des instructions complexes, avec une bonne adaptation au style formel attendu dans un contexte professionnel.
- **Avantage 2** : très bonne cohérence syntaxique et grammaticale, produisant des emails « prêts à l'envoi ».
- **Limitation** : tendance à uniformiser les réponses, avec des formulations parfois redondantes ou peu inventives sur plusieurs générations successives.

Mistral (open-mixtral-8x22b)

Modèle open-source récent, axé sur l'efficacité et la légèreté, utilisé via l'API de Mistral.ai :

- **Avantage 1** : diversité lexicale plus marquée, avec des tournures de phrases variées et parfois plus persuasives pour échapper au filtre.
- **Avantage 2** : génération rapide et infrastructure allégée, idéale pour des tests en boucle dans une logique ReAct.
- **Limitation** : sensibilité accrue au prompt : certaines réponses peuvent être moins cohérentes ou contenir des maladroresses si le prompt est mal calibré.

En résumé, GPT-3.5 est plus stable et sûr, tandis que Mistral favorise la créativité mais demande un meilleur contrôle des instructions.

4.1.3 Évaluation : détection par Naive Bayes

Chaque email est évalué par le classifieur Naive Bayes entraîné sur un jeu standard. On calcule le taux de détection :

$$P_{\text{spam}} = \frac{\text{Nombre d'emails détectés comme phishing}}{\text{Nombre total d'emails générés}} \quad P_{\text{bypass}} = 1 - P_{\text{spam}}$$

Exemple d'email généré

Exemple d'email généré par LLM (thème : business)

Sender : Brown and Sons Professional Development Team

Subject : Personalized Invitation for Edwin Lister — Exclusive Business Workshop

Dear Edwin Lister,

We hope this message finds you in good health and high spirits. At Brown and Sons, we are always seeking opportunities to help our valued employees grow professionally. As a specialist in your field, we believe that you would greatly benefit from attending our upcoming workshop, "*Mastering Business Strategies : A Comprehensive Approach.*"

This workshop is designed to provide you with the latest insights and techniques in the business world, as well as offer unique networking opportunities. Here are some key features you can expect :

- **Advanced Business Strategies :** Gain knowledge of cutting-edge business strategies from industry experts who have successfully navigated the market.
- **Hands-on Learning :** Engage in practical sessions and case studies to apply the newly acquired knowledge and enhance your skills.
- **Networking Opportunities :** Connect with like-minded professionals, potential mentors, and industry leaders to expand your professional network.
- **Exclusive Insights :** Receive valuable insights into emerging trends and future opportunities in the business world.

Date : [Insert Date]

Time : [Insert Time]

Venue : [Insert Venue]

To secure your spot in this exclusive workshop, please click the link below and complete the registration process :

<https://www.brownandsons-workshop.com>

Seats are limited, so we encourage you to register as soon as possible to avoid missing out on this fantastic opportunity.

We understand the importance of investing in your professional growth, and therefore, we have partnered with several reputable organizations to offer a special discount exclusively for professionals like yourself. By using the promo code PROF-SPECIALTY, you will receive a 15% discount on the workshop registration fee.

If you have any questions or need further information, please do not hesitate to reach out to our dedicated support team at support@brownandsons-workshops.com.

We look forward to welcoming you to this enriching workshop and witnessing your continued professional growth.

Best regards,

Brown and Sons Professional Development Team

*Remarque : certains éléments tels que la date, l'heure ou le lieu sont laissés vides ou génériques ([Insert Date], etc.). Cela reflète les **mécanismes de sécurité intégrés (guardrails)** dans les modèles de langage, conçus pour empêcher la génération d'emails de phishing pleinement réalistes ou utilisables directement.*

4.1.4 Analyse ciblée de l'exemple généré

L'email présenté ci-dessus illustre la capacité du modèle à s'adapter précisément aux caractéristiques du profil cible suivant :

- **Nom** : Edwin Lister
- **Âge** : 65 ans
- **Travail** : Professionnel dans une spécialité ("Prof-specialty")
- **Statut marital** : Marié ("Married-civ-spouse")
- **Sexe** : Homme
- **Entreprise** : Brown and Sons
- **Salaire** : >50K

On constate que l'email :

- reprend le nom complet ("Dear Edwin Lister"),
- cite l'entreprise "Brown and Sons" comme émettrice du message,
- cible explicitement un professionnel expérimenté en parlant de "specialist in your field" et d'un "exclusive business workshop",
- adopte un ton formel et valorisant, cohérent avec un homme de 65 ans occupant un poste à responsabilités,
- fait référence à des notions avancées de stratégie et de réseautage, pertinentes pour un haut niveau de revenu et d'ancienneté.

Ainsi, le prompt engineering permet ici une **personnalisation convaincante** en s'appuyant exclusivement sur des variables tabulaires extraites du dataset `adult.csv`.

4.2 Approche ReAct : Génération Adaptative

Principe

L'approche ReAct repose sur une boucle :

1. Génération d'emails,
2. Évaluation par le classifieur,
3. Conservation des emails non détectés,
4. Réinjection de ces exemples dans les prochains prompts.

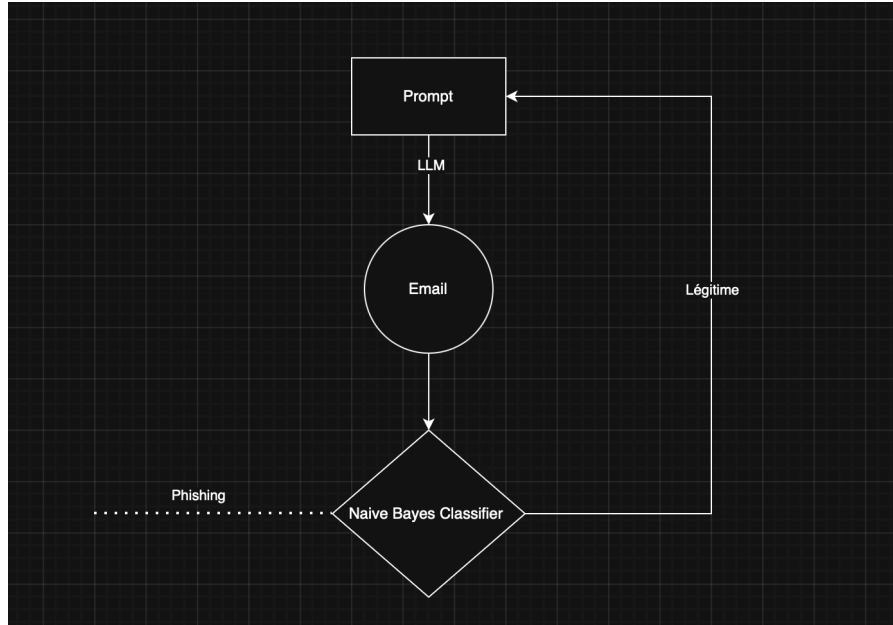


FIGURE 8 – Schéma de l’approche ReAct : les emails générés par le LLM sont évalués par un classifieur Naive Bayes. Les emails non détectés comme phishing (classés “légitime”) sont réinjectés dans le prompt pour guider les générations suivantes.

Prompt adaptatif avec exemples

Les meilleurs exemples (non détectés) sont sérialisés sous forme **few-shot** dans le prompt :

```

### Example 1
Target Profile:
- Name: Alice Smith
- Occupation: Accountant
Generated Email:
Dear Alice, your invoice has been flagged...
  
```

4.2.1 Évolution du taux de détection

Le modèle apprend progressivement à contourner le filtre Naive Bayes. Le graphique ci-dessous montre l’évolution du taux de spam détecté au fil des itérations :
Ce graphique met en évidence l’efficacité de la méthode ReAct dans le contournement progressif du classifieur. On observe que le **taux de détection décroît rapidement lors des premières itérations**, signe que les exemples non détectés utilisés comme *few-shots* permettent aux modèles de langage de générer des emails de plus en plus convaincants.

La courbe exponentielle ajustée $D_t = ae^{-bt}$, typique d’un processus de dégradation rapide suivi d’un palier, suggère une phase d’apprentissage efficace par le LLM à partir des premiers feedbacks. Cette tendance est cohérente avec une convergence vers une *taux plancher*, qui correspond à la robustesse résiduelle du filtre Naive Bayes.

- Les premières itérations permettent un gain rapide de performance (forte baisse du taux de détection).
- Les dernières itérations ont un effet marginal, témoignant de la saturation du modèle génératif vis-à-vis des stratégies d’évasion efficaces.

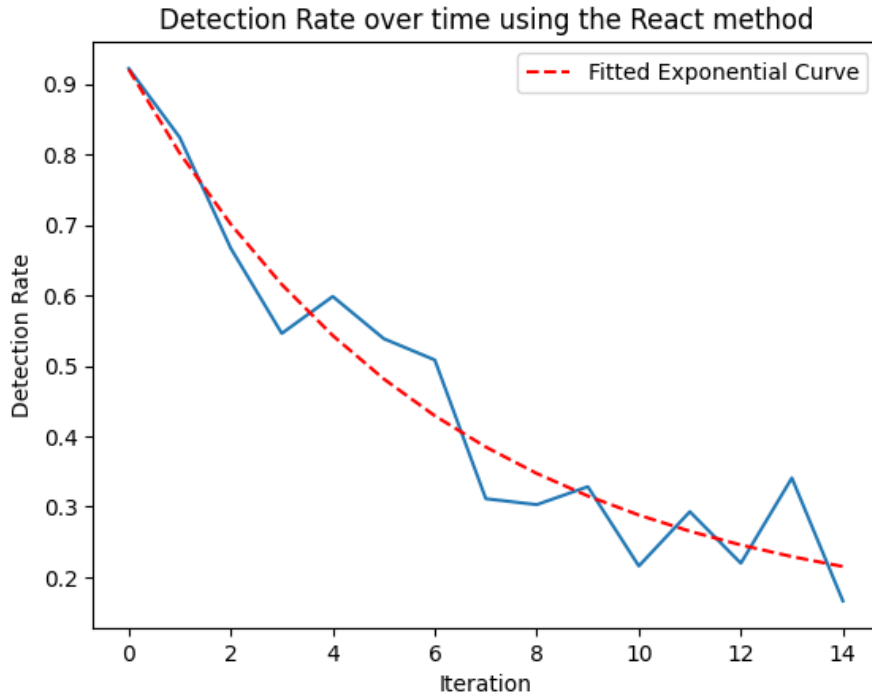


FIGURE 9 – Évolution du taux de détection au fil des itérations de la méthode ReAct, avec ajustement exponentiel. La courbe rouge représente un ajustement exponentiel de la forme $D_t = ae^{-bt}$, soulignant la décroissance progressive du taux de détection par le classifieur Naive Bayes.

- La méthode ReAct constitue donc une **stratégie adaptative redoutablement efficace** pour optimiser la génération d’emails adversariaux.

4.2.2 Limites

- **Biais d’adaptation au modèle Naive Bayes** : la méthode ReAct exploite le fait que Naive Bayes ne modélise que des distributions de mots indépendants. En retour, le modèle génératif apprend principalement à éviter les mots à forte log-probabilité conditionnelle associée au phishing. Cela optimise l’évasion du filtre sans réellement améliorer la plausibilité sémantique des emails générés, ce qui limiterait son efficacité face à un classifieur sémantique.
- **Risque de dérive lexicale** : à force d’éliminer les mots détectables, les emails générés peuvent s’appauvrir lexicalement ou devenir incohérents, perdant en crédibilité vis-à-vis d’un humain tout en contournant artificiellement le classifieur.
- **Absence de feedback supervisé** : le système repose uniquement sur un signal binaire (spam/non-spam), sans intégration de métriques de qualité linguistique, de diversité ou de vraisemblance. Cela rend l’optimisation unidimensionnelle et potentiellement fragile.

L’approche ReAct démontre qu’un LLM, même sans fine-tuning, peut apprendre à tromper un classifieur classique via prompt engineering adaptatif. Le taux de contournement augmente significativement au fil des itérations.

5 Génération de phishing par fine-tuning d'un LLM

5.1 Motivation du fine-tuning

Le **fine-tuning** consiste à adapter un modèle de langage pré-entraîné $\mathcal{M}_\theta$ (dans notre cas **Phi-2**), à une tâche spécifique en mettant à jour une partie ou l'ensemble de ses paramètres sur un jeu de données spécialisé. Contrairement au *prompt engineering*, qui guide les sorties du modèle à travers une formulation astucieuse des instructions d'entrée sans modifier les poids, le fine-tuning permet d'**encapsuler la tâche cible dans les poids du modèle lui-même**.

Ce mécanisme offre plusieurs avantages théoriques :

- **Réduction de la variance des sorties** : en s'appuyant sur une distribution apprise, le modèle produit des textes plus cohérents et moins sensibles aux variations subtiles du prompt, contrairement au prompting classique qui dépend fortement du sampling aléatoire.
- **Génération gratuite et autonome** : une fois le modèle fine-tuné, il peut être utilisé localement sans appel à une API externe (comme OpenAI ou Mistral), ce qui supprime les coûts liés à l'inférence et permet une génération à grande échelle de manière autonome.
- **Capacité à contourner des classifieurs lexicaux** : contrairement au prompt engineering qui opère sur des instructions visibles, le fine-tuning apprend une structure latente dans l'espace des textes, permettant de générer des séquences difficilement détectables par des modèles naïfs basés sur la fréquence des mots (e.g. Naive Bayes).
- **Réduction de la taille des entrées (prompts)** : une fois le modèle fine-tuné, il n'est plus nécessaire de fournir des exemples few-shot ni des consignes détaillées. Cela permet :
 - une génération plus rapide et économe en mémoire,
 - un traitement efficace à large échelle,
 - une meilleure compatibilité avec des modèles à fenêtre contextuelle limitée.

5.2 Constitution des données d'entraînement

Le jeu d'entraînement est constitué de couples (**prompt**, **phishing_email**), où :

- Le **prompt** est une instruction structurée décrivant le profil cible (issue du fichier **adult.csv** enrichi), dans le même format que celui utilisé pour le prompt engineering classique. Il inclut typiquement : âge, profession, sexe, salaire, entreprise, etc.
- Le **phishing_email** correspond à une sortie générée précédemment par un modèle LLM (GPT-3.5) à l'issue de la **méthode ReAct**, c'est-à-dire une génération ayant déjà réussi à contourner un classifieur entraîné.

Autrement dit, on entraîne le modèle à apprendre une fonction $f : \text{prompt} \mapsto \text{email}$ sur des exemples qui ont déjà passé avec succès le filtre de détection. Cela permet de consolider les motifs textuels les plus efficaces pour la tromperie, tout en autorisant une meilleure généralisation par interpolation.

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{L}_{\text{LM}}(\mathcal{M}_{\theta}(x), y)], \quad \text{où } \mathcal{L}_{\text{LM}} \text{ est la perte de type cross-entropie}$$

Ce choix de supervision directe par le résultat de la génération permet de dépasser les limitations lexicales du prompt engineering, en internalisant les régularités trompeuses

dans les poids du modèle.

5.3 Tokenization : passage du texte brut à l'entrée modèle

Les modèles de langage comme **Phi-2** ne manipulent pas directement des chaînes de caractères, mais des séquences d'**entiers naturels** correspondant à des tokens. La **tokenization** est l'étape de prétraitement qui mappe un texte T (chaîne de caractères) vers une séquence discrète $(t_1, \dots, t_n)$ où chaque $t_i \in \mathbb{N}$ est un indice dans le vocabulaire.

$$\text{Tokenizer} : T \mapsto (t_1, t_2, \dots, t_n) \quad \text{où } t_i \in \mathcal{V}, \mathcal{V} = \text{vocabulaire du modèle}$$

Pourquoi tokeniser ?

- Les modèles de type Transformer nécessitent une entrée indexée sous forme de tenseur $\mathbf{x} \in \mathbb{N}^n$;
- Le vocabulaire $\mathcal{V}$ permet de représenter efficacement des structures linguistiques fréquentes (mots, subwords, caractères, etc.) tout en limitant la taille du dictionnaire (souvent $|\mathcal{V}| \in [10^4, 10^5]$) ;
- La tokenisation préserve les régularités lexicales et morphologiques tout en assurant une compatibilité avec l'entraînement des embeddings.

Byte Pair Encoding (BPE) : principe et formulation

Le **Byte Pair Encoding** (BPE) est une méthode de tokenization qui vise à trouver une représentation sous forme de subwords pour un vocabulaire donné, tout en maintenant un dictionnaire de taille raisonnable. Ce procédé est utilisé par de nombreux modèles de langage modernes, y compris **Phi-2**.

Principe général. BPE repose sur une idée simple : remplacer de manière itérative les paires de symboles adjacents les plus fréquentes par un nouveau symbole.

1. Initialisation : on représente chaque mot du corpus comme une séquence de caractères terminée par un symbole spécial (par exemple “_”).
2. À chaque itération :
 - On compte toutes les paires de symboles adjacents dans le corpus (par exemple “t” suivi de “h” dans “the”).
 - On identifie la paire la plus fréquente (a, b) .
 - On la remplace dans tout le corpus par une nouvelle unité $[ab]$.
3. L'opération est répétée jusqu'à atteindre un nombre de merges fixé (typiquement plusieurs dizaines de milliers).

Avantages.

- Réduction de l'**out-of-vocabulary** : les mots inconnus peuvent toujours être décomposés en sous-unités fréquentes.
- Contrôle de la taille du vocabulaire (souvent $|\mathcal{V}| \approx 30\,000$ à $50\,000$).
- Meilleure robustesse face aux fautes de frappe, néologismes ou morphèmes rares.

Remarque. Le tokenizer de **Phi-2** est un tokenizer de type byte-level BPE : les symboles initiaux sont des octets, permettant un support universel sur l'ensemble des langues et jeux de caractères.

Exemple concret avec tokenizer BPE Soit le texte :

"Please confirm your identity to avoid account suspension."

Après passage par un tokenizer de type **Byte-Pair Encoding** , on peut obtenir :

"Please confirm your identity to avoid account suspension." (1)

$\mapsto$ [1032, 8721, 91, 11820, 27, 706, 11296, 18] (2)

où chaque entier représente un token appris automatiquement à partir des fréquences observées lors du pré-entraînement.

Remarque importante : Les tokens ne correspondent pas nécessairement à des mots entiers : par exemple, "suspension" peut être tokenisé en "susp" + "ension" si le mot complet n'est pas fréquent dans le vocabulaire.

Enfin, la tokenization permet de contrôler la longueur d'entrée (nombre de tokens n), un paramètre critique pour les modèles Transformer qui ont une complexité temporelle $\mathcal{O}(n^2)$ par couche :

- À chaque itération, on doit balayer l'ensemble du corpus pour **compter toutes les paires adjacentes** $\Rightarrow \mathcal{O}(n)$;
- On effectue jusqu'à n **fusions successives** (dans le pire cas où chaque paire est remplacée une seule fois) ;
- D'où une complexité totale de $\mathcal{O}(n^2)$.

Dans notre cas, nous utilisons le tokenizer associé au modèle **Phi-2**, basé sur un schéma **BPE**, avec un vocabulaire de taille $|\mathcal{V}| \approx 50\,000$ tokens. Il traite les textes à l'échelle du byte, ce qui le rend robuste aux langues multiples, aux fautes de frappe et aux variations de formatage.

5.4 Présentation du modèle Phi-2

Phi-2 est un modèle de langage auto-régressif développé par Microsoft, issu de la famille des Transformer déployés à des échelles réduites mais optimisées. Il s'agit d'un modèle open-source de type **decoder-only**, analogue à GPT-2/GPT-3, entraîné spécifiquement pour démontrer l'efficacité d'un entraînement rigoureux sur des corpus éducatifs filtrés.

Caractéristiques techniques.

- **Nombre de paramètres** : 2,7 milliards
- **Structure Transformer** avec attention causale uniquement (auto-régressive)
- **Fenêtre contextuelle** maximale de 2048 tokens
- **Pas de RLHF (reinforcement learning from human feedback)** pas de tuning avec rétroaction humaine, mais un entraînement supervisé sur des données de type textbook/code.

Forme fonctionnelle. Le modèle apprend une distribution conditionnelle sur les séquences de texte :

$$p_{\theta}(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t})$$

où chaque x_t est un token issu d'un vocabulaire $|\mathcal{V}| \approx 50,000$, et la prédiction est modélisée par un empilement de blocs Transformer :

$$\text{TransformerBlock}_\ell(h^{(\ell-1)}) = h^{(\ell)} = \quad (3)$$

$$\text{LayerNorm}(h^{(\ell-1)} + \text{MLP}(\text{LayerNorm}(h^{(\ell-1)} + \text{SelfAttention}(h^{(\ell-1)})))) \quad (4)$$

Où :

- $h^{(\ell-1)}$: représentation en entrée au bloc ℓ , de dimension d .
- **SelfAttention** : module d'attention multi-têtes causale, qui agrège l'information du passé :

$$\text{SelfAttention}(H) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

où $Q = HW^Q$, $K = HW^K$, $V = HW^V$ sont les requêtes, clés et valeurs, M est le masque causal.

- **MLP** : couche feed-forward appliquée indépendamment à chaque position :

$$\text{MLP}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2$$

avec $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$

- **LayerNorm** : normalisation des activations sur chaque vecteur $x \in \mathbb{R}^d$:

$$\text{LayerNorm}(x) = \frac{x - \mu(x)}{\sigma(x)} \cdot \gamma + \beta$$

avec $\mu(x)$ et $\sigma(x)$ la moyenne et l'écart-type, et γ, β des paramètres appris.

Motivation du choix. Phi-2 a été retenu dans notre projet pour plusieurs raisons :

- Son **rapport qualité / coût computationnel** est excellent : il atteint des performances proches de LLaMA-2 7B sur certains benchmarks, avec une taille 3 fois inférieure.
- Il peut être **entraîné ou fine-tuné efficacement** sur GPU accessibles (A100, T4) avec des techniques comme LoRA ou QLoRA.
- Sa licence open-source permet une intégration libre et reproductible dans un cadre académique.

Positionnement. En termes de taille, Phi-2 est comparable à des modèles comme Falcon 1B, GPT-NeoX 2.7B ou Mistral 2B, mais avec une **optimisation du corpus d'entraînement** qui améliore sa capacité à raisonner logiquement et à suivre des instructions simples.

5.5 Fine-tuning efficace avec QLoRA 4-bit

Le fine-tuning complet d'un modèle de langage comme Phi-2 (2,7B de paramètres) reste très coûteux, même avec des GPU modernes. Pour remédier à cette limitation, nous utilisons la méthode **QLoRA** (Quantized Low-Rank Adaptation), proposée par [4], qui combine trois techniques complémentaires :

- **Quantization 4-bit** des poids du modèle de base ;
- **Fine-tuning LoRA** sur les matrices clés (q_proj, v_proj) ;
- **Offloading mémoire optimisé** (paged optimizers, gradient checkpointing).

1. Quantization 4-bit. On applique une **quantification post-entraînement** des poids $\theta \in \mathbb{R}^d$ du modèle pré-entraîné, avec une précision de 4 bits par poids. Mathématiquement, cela revient à approximer chaque poids par :

$$\theta_i \approx \tilde{\theta}_i = \text{scale}_i \cdot q_i, \quad q_i \in \{-8, -7, \dots, 7\}$$

où q_i est un entier sur 4 bits, et scale_i est un facteur de redressement appris ou estimé par bloc. Cette compression réduit la mémoire GPU par un facteur $\times 2$ à $\times 4$ selon l'architecture.

2. Injection de LoRA. Les poids gelés quantifiés sont complétés par des modules LoRA, de la forme :

$$W_{\text{eff}} = \tilde{W} + \Delta W = \tilde{W} + AB$$

où :

- $\tilde{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$: poids du modèle pré-entraîné quantifié à 4 bits (et non mis à jour).
- $\Delta W = AB \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$: correction entraînable de rang faible ajoutée à $\tilde{W}$, où :
 - $A \in \mathbb{R}^{r \times d_{\text{in}}}$ (projection descendante),
 - $B \in \mathbb{R}^{d_{\text{out}} \times r}$ (projection montante),
 - avec $r \ll d_{\text{in}}, d_{\text{out}}$.

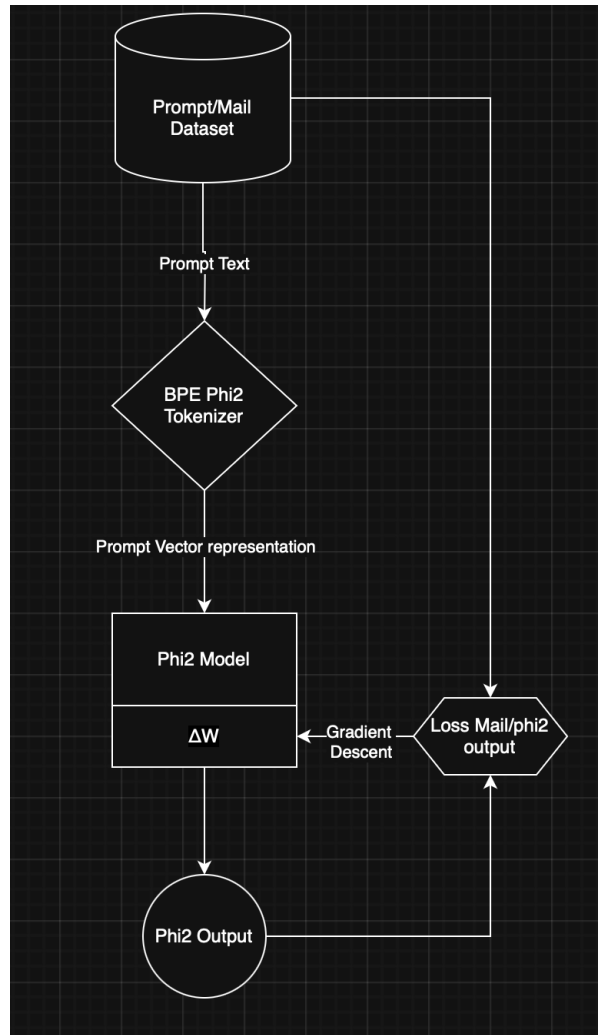
Seuls A et B sont appris pendant le fine-tuning, ce qui permet :

- de ne pas rétropropager dans les poids quantifiés,
- d'utiliser un optimizer classique (Adam) sur un sous-espace très réduit.

3. Optimisation mémoire. La combinaison des techniques suivantes permet d'effectuer le fine-tuning sur un GPU de 24–40 Go :

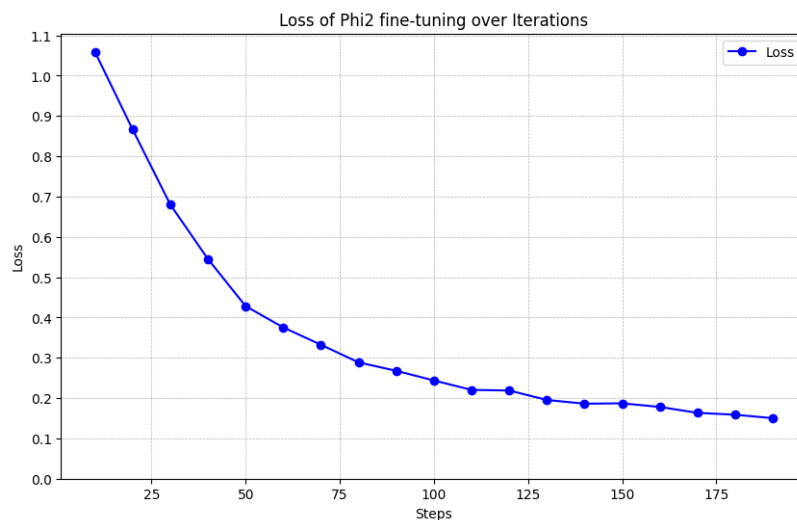
- **Paged optimizers** : évitent de charger tous les gradients en mémoire ;
- **Gradient checkpointing** : recalcul des activations à la volée pour économiser la RAM ;
- **Mixed precision training** (FP16/FP32) pour les poids LoRA.

Schéma récapitulatif.



5.6 Résultats du fine-tuning et impact sur la détection

Nous présentons ici les résultats obtenus à l'issue du fine-tuning du modèle Phi-2 avec QLoRA 4-bit sur notre jeu de données. Le graphique suivant illustre la décroissance progressive de la **loss d'entraînement** au fil des itérations :



On observe une convergence stable à partir de 150 steps, avec une perte qui passe de 1.06 à 0.15, témoignant d'un apprentissage progressif mais efficace des patterns de phishing à partir des prompts courts.

6 Conclusion

Ce projet a permis de montrer comment des modèles de langage peuvent être utilisés non seulement pour générer des e-mails de phishing crédibles, mais aussi pour tester la robustesse des systèmes de détection classiques. Dans un premier temps, nous avons mis en place un classifieur Naive Bayes, simple mais performant, capable de détecter efficacement les spams sur des données similaires à l'entraînement, avec une précision de 97,6%.

Nous avons ensuite exploré la génération d'e-mails de phishing via *prompt engineering*, couplée à une boucle adaptative ReAct. En quelques itérations, le taux de détection des e-mails générés est passé de 90% à 15%, illustrant la capacité des modèles GPT et Mistral à apprendre à contourner dynamiquement les filtres bayésiens. Ces résultats montrent que même sans apprentissage supervisé, les LLMs peuvent intégrer des stratégies efficaces d'évitement à partir de simples signaux de retour binaire.

Enfin, le fine-tuning du modèle Phi-2 avec QLoRA sur des couples (**prompt**, **email**) validés a permis d'encapsuler directement ces structures d'évitement dans les poids du modèle. Le taux de détection moyen des e-mails générés par ce modèle fine-tuné tombe à 22%, ce qui démontre l'efficacité de cette approche par rapport à la génération naïve.

Nous considérons ces deux approches — prompt engineering et fine-tuning — comme complémentaires : la première est particulièrement adaptée à des usages locaux ou exploratoires, tandis que la seconde permet un déploiement plus robuste à long terme, notamment pour une génération autonome et à grande échelle.

Ces résultats soulignent la vulnérabilité des filtres lexicaux face à des attaques générées dynamiquement, et plus largement la nécessité d'une redéfinition des stratégies défensives face aux modèles génératifs. En perspective, il serait pertinent de tester la robustesse des contenus générés face à un **classifieur sémantique**, capable de détecter des intentions cachées ou des structures rédactionnelles suspectes. Des modèles comme DeBERTa, RoBERTa ou DistilBERT fine-tunés sur des jeux récents de phishing pourraient constituer une base de comparaison intéressante. De même, l'utilisation de bases de données d'e-mails plus réalistes et contemporaines, collectées en entreprise, permettrait d'évaluer de manière plus fiable la transférabilité et la dangerosité des textes générés.

Références

- [1] Khalifa AFANE et al. “Next-Generation Phishing : How LLM Agents Empower Cyber Attackers”. In : 2024 IEEE International Conference on Big Data (BigData). IEEE. 2024, p. 2558-2567.
- [2] Minhaz F. Zibran ARIFA I. CHAMPA Md Fazle Rabbi. Phishing Email Curated Datasets. Les jeux de données couvrent des e-mails de 1995 à 2022. Voir également : - Champa, A. I., Rabbi, M. F., Zibran, M. F. (2024). ”Why Phishing Emails Escape Detection : A Closer Look at the Failure Points”. In 12th International Symposium on Digital Forensics and Security (ISDFS), pp. 1-6. IEEE. - Champa, A. I., Rabbi, M. F., Zibran, M. F. (2024). ”Curated Datasets and Feature Analysis for Phishing Email Detection with Machine Learning”. In 3rd IEEE International Conference on Computing and Machine Intelligence (ICMI), pp. 1-7. IEEE. Accédé le 8 mai 2025. 2024. URL : <https://zenodo.org/records/8339691>.
- [3] Akshaya ARUN et Nasr ABOSATA. “Next Generation of Phishing Attacks using AI powered Browsers”. In : arXiv preprint arXiv :2406.12547 (2024).
- [4] Tim DETTMERS et al. “QLoRA : Efficient Finetuning of Quantized LLMs”. In : arXiv preprint arXiv :2305.14314 (2023). Extended NeurIPS submission. URL : <https://arxiv.org/abs/2305.14314>.
- [5] Julian HAZELL. “Spear phishing with large language models”. In : arXiv preprint arXiv :2305.06977 (2023).
- [6] R KARANJAI. “Targeted phishing campaigns using large scale language models. arXiv”. In : arXiv preprint arXiv :2301.00665 (2022).
- [7] RED HAT. LoRA vs. QLoRA. LoRA et QLoRA sont deux techniques de fine-tuning efficaces pour les LLMs. Consulté le 14 mai 2025. Fév. 2025. URL : <https://www.redhat.com/fr/topics/ai/lora-vs-qlora>.
- [8] SCIKIT-LEARN DEVELOPERS. 1.9. Naive Bayes. Consulté le 6 mai 2025. scikit-learn. 2025. URL : https://scikit-learn.org/stable/modules/naive_bayes.html#weights-calculation.
- [9] Shrawan Kumar TRIVEDI. “A study of machine learning classifiers for spam detection”. In : 2016 4th International Symposium on Computational and Business Intelligence (ISCBI 2016), p. 176-180. DOI : [10.1109/ISCBI.2016.7743279](https://doi.org/10.1109/ISCBI.2016.7743279).
- [10] Shunyu YAO et al. “ReAct : Synergizing Reasoning and Acting in Language Models”. In : arXiv preprint arXiv :2210.03629 (2022). Published as a conference paper at ICLR 2023. URL : <https://arxiv.org/abs/2210.03629>.

7 Annexe

7.1 Statistiques Descriptives

7.1.1 Nombre Moyen de Liens Hypertexte par E-mail

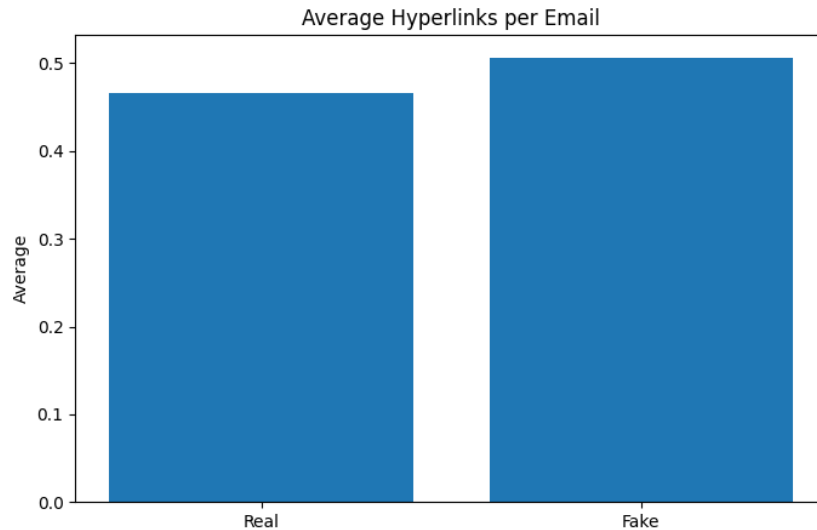


FIGURE 10 – Nombre moyen de liens hypertexte dans les e-mails réels et faux.

Les e-mails de phishing contiennent en moyenne un peu plus de liens hypertexte que les e-mails réels. Cela peut s'expliquer par une stratégie courante du spam visant à rediriger l'utilisateur vers des sites externes, souvent à des fins malveillantes (hameçonnage, fraude, publicité intrusive).

7.1.2 Distribution de la longueur des e-mails

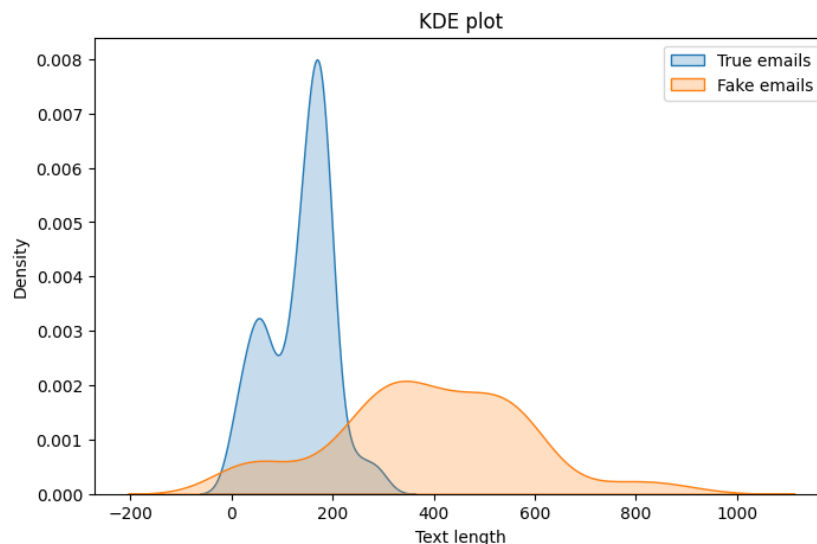


FIGURE 11 – Comparaison des longueurs des e-mails réels et faux.

On observe que les faux e-mails sont plus variés en termes de longueur. Cela peut refléter la tentative des attaquants de s'adapter à différents contextes ou scénarios. Certains faux e-mails peuvent être très courts et directs, tandis que d'autres peuvent être

plus longs et plus élaborés pour tromper davantage les destinataires. À l'inverse, les e-mails légitimes semblent plus uniformes en longueur, probablement en raison de formats plus standardisés.

7.2 Comparaison à la Loi de Zipf

La loi de Zipf stipule que la fréquence f d'un mot est proportionnelle à $1/r$, où r est son rang dans la liste des mots, triée par fréquence. Mathématiquement, cette relation se présente sous la forme suivante :

$$f(r) \propto \frac{1}{r} \quad (5)$$

où :

- r est le rang du mot (1 pour le mot le plus fréquent, etc.)
- $f(r)$ est la fréquence du mot de rang r

Ansi, quelques mots très fréquents dominent le texte tandis que la majorité des mots apparaissent rarement.

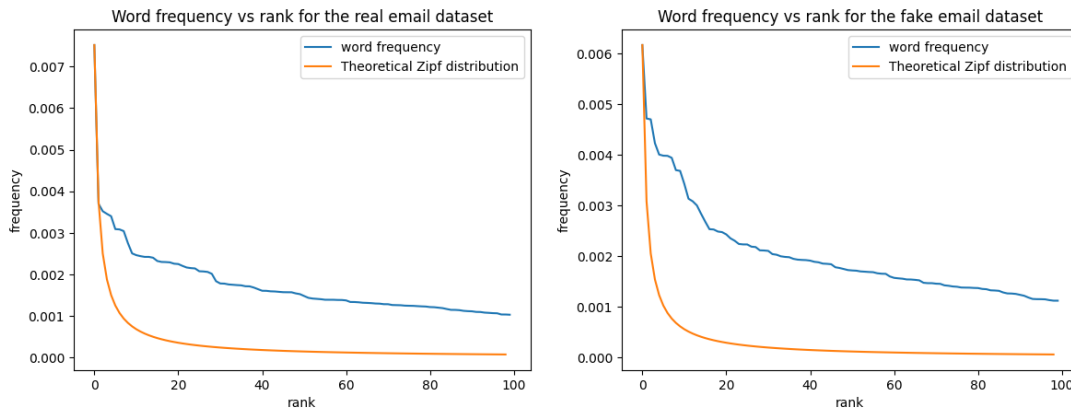


FIGURE 12 – Fréquence des mots (sans stopwords) comparée à la loi de Zipf.

On observe que les e-mails réels suivent plus fidèlement la loi de Zipf, tandis que les e-mails faux présentent un écart plus marqué, notamment pour les mots les plus fréquents. Cela peut indiquer une structure linguistique plus artificielle dans les spams, avec une répétition anormalement élevée de certains termes accrocheurs ou de persuasion, utilisés pour maximiser leur impact.

7.3 Emploi de Naive Bayes dans l'identification des spams

Dans le contexte de la détection de spams [8], les classes sont généralement $c \in \{\text{spam}, \text{ham}\}$, et les caractéristiques x_i représentent la fréquence ou la présence de mots spécifiques dans le courriel. Le modèle multinomial est particulièrement adapté, car il considère le nombre d'occurrences de chaque mot.

Les probabilités $P(c)$ sont estimées par la proportion de documents appartenant à la classe c dans l'ensemble d'entraînement. Les probabilités conditionnelles $P(w_i | c)$, où w_i est un mot du vocabulaire, sont estimées par :

$$P(w_i | c) = \frac{N_{w_i, c} + \alpha}{N_c + \alpha \cdot V}$$

où :

- $N_{w_i,c}$ est le nombre d'occurrences du mot w_i dans les documents de la classe c ,
- N_c est le nombre total de mots dans les documents de la classe c ,
- V est la taille du vocabulaire (nombre total de mots distincts),
- α est le paramètre de lissage de Laplace (généralement $\alpha = 1$).

7.4 Application Streamlit

The screenshot shows a web application titled "Génération et Détection d'Email de Phishing". It has three tabs: "Détection d'Email", "Génération Individuelle" (which is active), and "Génération de Groupe". Under the "Génération d'Email Individuel" tab, there is a dropdown menu for "Langue de l'email" set to "français". Below this is a section for uploading a JSON file, with a "Browse files" button. Further down, there is a section titled "Ou entrez les caractéristiques manuellement :" with input fields for "Nom", "Âge" (set to 0), "Localisation", and "Description". A "Générer un email" button is at the bottom.

FIGURE 13 – Interface Streamlit pour la génération d’e-mails de phishing à partir de profils personnalisés.